
Exercise sheet 5 for Numerical Optimization

Dr. Michael Lehn

Tobias Born

Deadline: November 19, 2025

Exercise 1 (Direct search methods - Nelder-Mead method, 1+7+1+1 points)

The Nelder-Mead method is another direct search method. For $f : \mathbb{R}^n \rightarrow \mathbb{R}$ an n -dimensional polytope consisting of $n+1$ vertices is moved across the domain according to specific rules in such a way that, hopefully, it will closely enclose the local minimum at the end. Iteratively, the vertex of the polytope with the highest function value is replaced. The new vertex is determined using four operations: reflection, expansion, contraction, shrinking. Depending on the behaviour of the function, one of these operations is performed in each iteration step.

- a) The detailed procedure and algorithm can be found in the lecture notes. Read this chapter and understand why each of the four operations is performed in which situation.
- b) Implement the method. Therefore, fill in the gaps in `nelder_mead.m`.
- c) The MATLAB programme `test_nelder_mead_rosenbrock.m` applies your implementation to Rosenbrock's function using a random initial polytope. Execute it and reflect on the results.
- d) The MATLAB programm `test_nelder_mead_himmelblau.m` does the same to Himmelblau's function. Execute it and reflect on the results.

Exercise 2 (Descent methods with line search, 6+6+3+3+3+3+3 points)

After dealing with direct search methods, we will now examine a new class of approaches, *descent methods*. These require the optimization problem to satisfy more conditions, namely that f is continuously differentiable. Descent methods generate a sequence of iterates

$$x_0, x_1, x_2, \dots \in \mathbb{R}^n .$$

The $(k+1)$ -st iterate x_{k+1} arises from the k -th iterate x_k as

$$x_{k+1} = x_k + \alpha_k d_k ,$$

where

- i) the search direction $d_k \in \mathbb{R}^n$ with $\|d_k\|_2 = 1$ is chosen such that f , starting from x_k , decreases in the direction d_k .
- ii) The step size $\alpha_k \in \mathbb{R}_+$ specifies how far from x_k in the direction of d_k the next iterate x_{k+1} lies.

i) Search direction: A vector $d_k \in \mathbb{R}^n$ with $\|d_k\|_2 = 1$ is called *direction*. A direction d_k is called a *search direction* if the directional derivative of f at x_k in direction d_k is negative, so

$$\nabla f(x_k)^T d_k < 0 .$$

That is because then there exists an interval of step sizes $(0, \tilde{\alpha})$ with $\tilde{\alpha} > 0$, such that for every $\alpha_k \in (0, \tilde{\alpha})$ it is

$$f(x_{k+1}) = f(x_k + \alpha_k d_k) < f(x_k) ,$$

and it is worthwhile to use this direction to determine x_{k+1} and to search for a suitable step size.

- a) Proof that if a direction $d_k \in \mathbb{R}^n$ fulfils $\nabla f(x_k)^T d_k < 0$, there is an interval of step sizes $(0, \tilde{\alpha})$ with $\tilde{\alpha} > 0$ such that for $\alpha_k \in (0, \tilde{\alpha})$ it is $f(x_k + \alpha_k d_k) < f(x_k)$.

For $\nabla f(x_k) \neq \mathbf{0}$, the specific direction

$$d_k = -\frac{\nabla f(x_k)}{\|\nabla f(x_k)\|_2} \tag{1}$$

is called *steepest descent direction*, as

$$\nabla f(x_k)^T \left(\frac{-\nabla f(x_k)}{\|\nabla f(x_k)\|_2} \right) = \min_{\substack{d \in \mathbb{R}^n \\ \|d\|_2=1}} \nabla f(x_k)^T d . \tag{2}$$

- b) Prove (2) and prove also that (1) is the unique minimizer.

Important special cases of search directions can be written in the form

$$d_k = \frac{-B_k \nabla f(x_k)}{\|B_k \nabla f(x_k)\|_2} \quad \text{with} \quad B_k \in \mathbb{R}^{n \times n} .$$

For example,

- $B_k = I$ yields the *gradient descent method* with $d_k = \frac{-\nabla f(x_k)}{\|\nabla f(x_k)\|_2}$, so the direction of the steepest descent (see 1).
- For $B_k = (H_f(x_k))^{-1}$ one gets Newton's method.

ii) Determination of the step size: Now let a search direction d_k be chosen. The question then is how far to go in this direction, i.e., how large α_k should be. The intuitive idea would probably be to choose the point on the line through x_k in the direction of d_k that minimizes the function value on the line, i.e., to choose α_k such that

$$f(x_k + \alpha_k d_k) = \min_{\alpha \in \mathbb{R}_+} f(x_k + \alpha d_k) . \quad (3)$$

Unfortunately, this is not possible in general, as (3) would just be another optimization problem we would have to solve. Of course, for special cases of functions, e.g. quadratic ones, you can carry out an exact line search.

In practice, one looks for easily verifiable conditions on the step size so that the resulting decrease in the objective function is acceptably large. At first the *Armijo condition*. For $f : \mathbb{R}^n \rightarrow \mathbb{R}$ continuously differentiable and a constant $c_1 \in (0, 1)$, a step size $\alpha \in \mathbb{R}^n$ fulfils the Armijo condition if

$$f(x_k + \alpha d_k) \leq f(x_k) + c_1 \alpha \nabla f(x_k)^T d_k . \quad (4)$$

- c) Using a simple one-dimensional function, think about what the Armijo condition (4) actually means and what c_1 does.

The problem with the Armijo condition is that it is satisfied by every sufficiently small step size. The *curvature condition* is thought to prevent this. It demands that

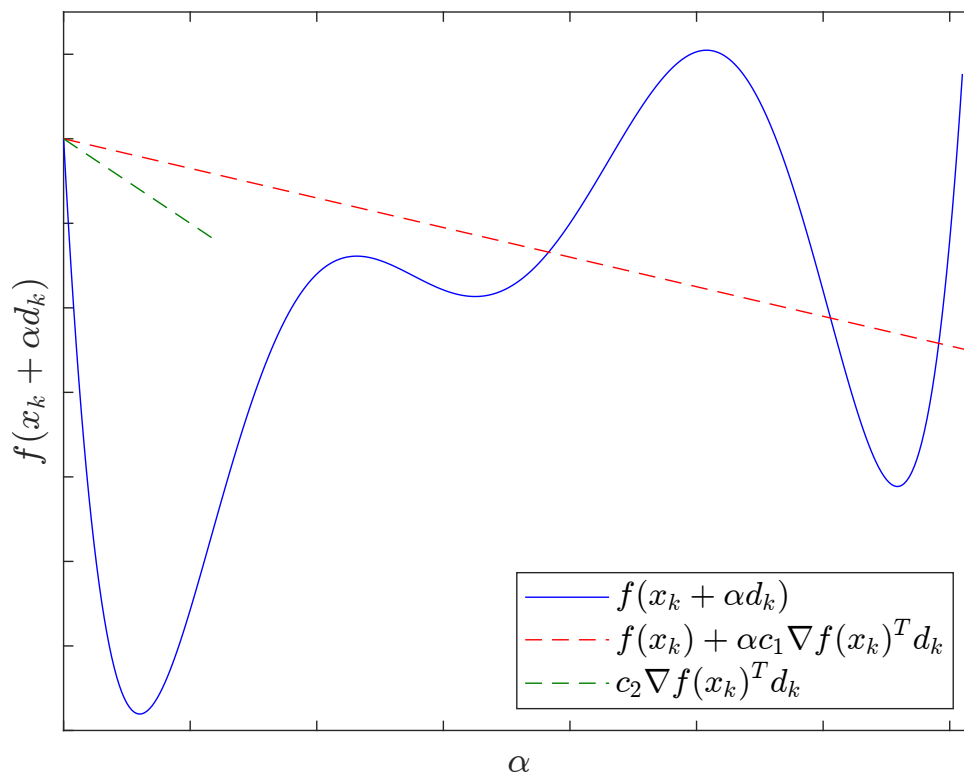
$$\nabla f(x_k + \alpha d_k)^T d_k \geq c_2 \nabla f(x_k)^T d_k \quad (5)$$

with a constant $c_2 \in (c_1, 1)$.

- d) Using a simple one-dimensional function, think about what the curvature condition (5) actually means and what c_2 does.

The combination of both, the Armijo condition and the curvature condition, is known as *Wolfe conditions*.

- e) Look at the function graph in the following figure. Where are which of the above conditions met?



A problem of the curvature condition is that it allows $\nabla f(x_k + \alpha d_k)^T d_k$ to have a large value. Therefore, the *strong Wolfe conditions* demand an even stronger condition on the curvature. The Armijo condition remains the same, but the curvature condition is replaced by

$$|\nabla f(x_k + \alpha d_k)^T d_k| \leq c_2 |\nabla f(x_k)^T d_k| \quad (6)$$

with some constant $c_2 \in (c_1, 1)$.

- f) Using a simple one-dimensional function, think about what the condition (6) actually means.
- g) Consider the figure from exercise e). Where are the strong Wolfe conditions met?

We defined the step size rules differently on the exercise sheet than in the lecture notes. Here, we defined a general condition, (4), for Armijo step sizes that can be satisfied by multiple α . In practice, the question now arises as to how an Armijo step size can be determined in a meaningful way. The aim is to get a step size that fulfils the Armijo condition, is as large as possible and at the same time it is not a lot of effort required to determine the step size. A possible approach is to set a maximum step size and reduce it until the Armijo condition is satisfied. And that is exactly how the Armijo step size is defined in the lecture notes. There, the Armijo step size is a unique step size and its definition directly specifies a procedure for its determination.

Exercise 3 (Implementation of a general descent method, 5+5+1+2 points)

We want to implement a general descent method so that the search direction and the step size determination can be passed as function handles.

- a) Create a MATLAB program `descent_method.m` that implements a general descent method. It should receive a starting point `x0`, function handles
- `grad_f(x)` for the function's gradient,
 - `direction(x)` for the search direction and
 - `step_size(x,d)` for the step size,

where `x` is a point in the domain and `d` is the respective search direction, as well as a maximum number of iteration steps `max_it` and a tolerance `tol`.

It is supposed to terminate when the norm of the function's gradient is smaller than the tolerance or the maximum number of iteration steps is reached. It is supposed to return a vector of dimension $(2 \times (\text{number iteration steps} + 1))$ containing all iterates.

We want to test two choices of step sizes, in particular the Armijo step size.

- b) Create a MATLAB program `armijo_step_size.m` that determines the unique Armijo step size as defined in the lecture notes. Actually, its definition directly gives the procedure to compute it. The program is supposed to receive the current iterate `x`, the search direction `d`, a function handle `f`, a function handle `grad_f` and the two parameters `beta` and `sigma` as defined in the lecture notes. It should return the Armijo step size.

We want to test two different search directions, namely the *gradient descent method* and *Newton's method*, on the examples of Rosenbrock's function and Himmelblau's function. We want to try as well the Armijo step size as a constant step size of one.

- c) The MATLAB program `test_descent_methods_rosenbrock.m` performs your descent method with all four combinations of search directions and step sizes. Fill the gaps in the function handles for the search directions. For the gradient descent, normalize the length of the search direction to one before determining the step size. For Newton's method, do not do that. Then execute the program and reflect on the outcome.
- d) Do the same with the MATLAB program `test_descent_methods_himmelblau.m`, which considers Himmelblau's function. Execute it and reflect on the outcome. Newton's method in combination with the Armijo step size stays at `x0` for many random starting points. Why is this the case?